

FPAS: Frontier-Based Path Planning with Adaptive Sampling for Large-Scale Unknown Environments

Jinwoo Choi, Yeonkyu Lee, Jung-Taak Kim, Jisung Bae, Seung-Woo Seo

Abstract—In this work, we propose Frontier-based Path Planning with Adaptive Sampling (FPAS), a novel framework designed for efficient goal-reaching in large-scale, unknown environments. While existing planners often struggle with computational bottlenecks or inefficient paths during long-range navigation, FPAS overcomes these challenges by reinterpreting the frontier concept for goal-directed tasks. Specifically, our method leverages frontiers to effectively guide forward progression into unobserved regions and to select promising subgoals for backtracking from dead-ends or inefficient paths. Furthermore, FPAS introduces an adaptive sampling mechanism based on a frontier-derived openness metric. This mechanism dynamically adjusts the global graph’s density by employing sparse nodes in open areas to alleviate computational burdens, while preserving denser sampling in narrow passages to ensure connectivity. Extensive evaluations demonstrate that FPAS substantially improves computational efficiency over baseline methods while maintaining highly competitive goal-reaching performance.

I. INTRODUCTION

Generating paths to reach goals in large-scale unknown environments is a challenging task that has become increasingly relevant in various domains, including autonomous driving [1], robotics [2], search and rescue missions [3], and military operations [4]. The primary objective is to find a goal-directed path that minimizes travel cost and time under limited observations, while remaining robust to uncertainty in the environment. Because the map is only partially known, the planner must continuously adapt its decisions to newly revealed space while efficiently managing the growing volume of accumulated spatial data.

Despite active research, existing path generation approaches often struggle to meet these demands and fall short when reaching goals in large-scale unknown environments. These methods can be broadly categorized into search-based, sampling-based, and visibility-graph-based approaches. While search-based methods (e.g., A* [5], D* [6]) offer strong optimality, their search costs scale poorly when frequent plan revisions and continuous map updates are required. Sampling-based methods like RRT are highly applicable in continuous spaces. However, their inherent randomness and goal-directed expansion heuristics often cause unnecessary detours or back-and-forth behavior in complex terrains. Recently, visibility-graph-based methods,

This research was supported by the Challengeable Future Defense Technology Research and Development Program through the Agency for Defense Development (ADD) funded by the Defense Acquisition Program Administration (DAPA) in 2026 (No. 915108201). (Corresponding author: Seung-Woo Seo)

The authors are with the Department of Electrical and Computer Engineering, Seoul National University, Seoul, South Korea (email: {wlsdn9350, yeonkyu011, mychoco333, trr0631, sseo}@snu.ac.kr)

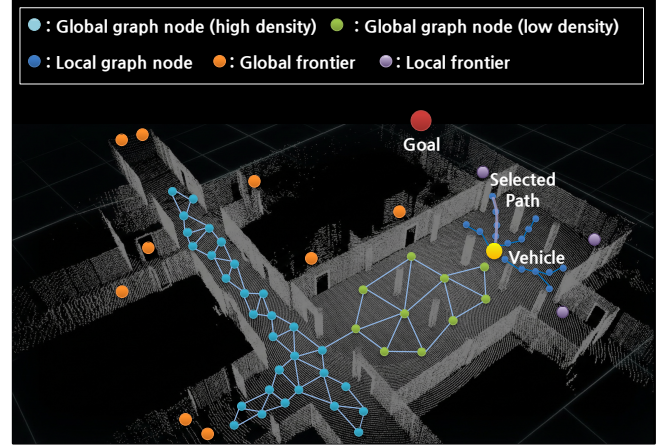


Fig. 1: **Overview of the approach.** Our method navigates the vehicle toward the goal without prior knowledge of the unexplored area. The reactive planning stage in our method generates a path guided by the local frontier within the sensor range. The replanning stage generates a path to the subgoal through the global graph and global frontier. The global graph is generated with low node density in open areas and high node density in narrow areas through adaptive sampling.

notably the FAR Planner [7], have emerged as powerful alternatives in unknown environments. Yet, as the number of vertices increases, the cost of checking visibility connections and updating the graph escalates quickly. Consequently, in scenarios where the graph expands due to complex obstacle distributions or increased exploration time, computational overhead remains a significant bottleneck. Therefore, a novel framework is critical to appropriately balance goal-reaching with the exploration of unobserved regions, while efficiently managing the memory of environmental information.

In this paper, we propose Frontier-based Path Planning with Adaptive Sampling (FPAS), a novel path generation framework designed for efficient goal-reaching in large-scale, unknown environments (Figure 1). The core insight of FPAS is the reinterpretation of the *frontier* concept, which is traditionally utilized in exploration tasks [8] for the goal-reaching problem. Specifically, FPAS leverages frontiers for two primary purposes: guiding navigation and adaptively managing the global graph. The proposed framework operates through two distinct stages. First, the reactive planning stage advances into unobserved regions within the sensor range, where frontiers dictate a forward direction highly advantageous for both goal-reaching and exploration. Sec-

ond, the replanning stage backtracks from dead-ends or inefficient paths, where frontiers serve as promising subgoals to determine exactly where the robot should retreat for a new search. Furthermore, driven by our key intuition that the required graph density varies with local spatial structure, we introduce an adaptive sampling mechanism based on a local “openness” metric, which is directly derived from the spatial characteristics of the frontiers. This mechanism dynamically adjusts node density by employing sparse nodes in open areas to reduce computational burdens, while preserving denser sampling in narrow passages to ensure collision avoidance and connectivity. Ultimately, FPAS sustains high goal-reaching performance while drastically decreasing computational demands.

The main contributions of this paper are threefold. First, we reinterpret the frontier concept to propose a path generation strategy that effectively guides forward progression and selects promising subgoals for backtracking. Second, we introduce an adaptive sampling mechanism based on a frontier-derived openness metric, which dynamically adjusts the global graph’s density to significantly alleviate storage and computational burdens. Finally, extensive evaluations demonstrate that FPAS maintains competitive goal-reaching performance while substantially improving computational efficiency over baseline methods.

II. RELATED WORK

A. Goal-reaching path planners

Existing path planning algorithms for goal-reaching can be broadly classified into three main categories: search-based, sampling-based, and visibility-graph-based approaches.

Search-based planners, such as A* [5] and D* [6], are widely used in path planning. However, standard A* is inefficient in unknown environments because it requires prior knowledge and replans from scratch. To address this, incremental search algorithms like Lifelong Planning A* (LPA*) [9], D*, and its streamlined successor D* Lite [10] enable efficient replanning by reusing previous search results. Furthermore, hierarchical approaches [11] enhance efficiency by organizing planning across multiple abstraction levels. Despite these refinements, their fundamental reliance on dense grids or graphs causes search costs to scale poorly in large-scale, unknown environments requiring frequent map updates.

Random sampling-based planners have gained considerable attention in motion planning, particularly for handling high-dimensional and complex environments. Among these, Rapidly-exploring Random Trees (RRTs) are the most prominent, leading to numerous extensions such as bidirectional RRT [12], informed RRT* [13], and kinodynamic RRT* [14]. To address dynamic or partially known settings, RRT^X [15] introduced a fast replanning approach that adapts to newly recognized information, while recent methods like FHQ-RRT* [16] employ KeyPoints node creation to accelerate high-quality path generation. Despite these advancements, their inherent randomness and goal-directed heuristics often cause unnecessary detours and susceptibility to local

minima in complex terrains. Furthermore, the high computational cost of continuously updating the tree restricts their scalability in large-scale unknown environments [17], [18].

Visibility-graph-based approaches generate paths by connecting mutually visible vertices, a foundational concept rooted in the Probabilistic Roadmap Method (PRM) [19]. Building upon this, the FAR Planner [7] has recently emerged as a powerful alternative for goal-reaching in unknown environments, utilizing a polygon-based environment model and dynamic visibility updates. While highly effective, these approaches face two major limitations. First, formalizing irregular obstacles into polygons is often difficult, which decreases the planner’s generalizability in unstructured environments. Second, as the graph expands due to complex obstacle distributions or increased exploration time, the cost of checking visibility connections escalates quickly. Consequently, this computational overhead remains a significant bottleneck in large-scale applications.

To overcome the limitations of these prior studies, our proposed FPAS framework aims to appropriately balance goal-reaching with the exploration of unobserved regions, while efficiently managing the memory of environmental information.

B. Next-best-view approaches

Next-best-view (NBV) approaches are designed to optimize the coverage of an entire environment by quantifying the information that can be obtained from each viewpoint and generating paths that maximize it. Previous research, such as NBVP [20], employed the RRT algorithm to generate viewpoints and paths. However, this approach focused only on locally expanding the boundaries of the exploration area without considering the overall environment, leading to difficulties in long-term exploration. To overcome this limitation, subsequent studies such as GBP [21] and DSVP [8] have proposed a dual-stage exploration approach. This approach comprises two stages: one stage focuses on exploring areas within the sensor range, while the other stage involves moving to the explored areas. DSVP, in particular, utilized the boundary between known and unknown areas, known as the frontier, to guide the expansion of RRT. They also proposed a dynamically-expanded RRT that only maintained nodes in free space within the recognition range, thereby reducing computation. Building upon DSVP, we extend its frontier-based strategy to goal-reaching tasks by utilizing frontiers as subgoal candidates. Furthermore, we propose a novel objective function that balances exploration and exploitation, alongside an adaptive sampling method for efficient graph management. Ultimately, these advancements enable highly efficient path generation for goal-reaching in large-scale, unknown environments.

III. METHODOLOGY

In this section, we present the Frontier-based Path Planning with Adaptive Sampling (FPAS) framework. A key idea of FPAS is to repurpose exploration frontiers as goal-reaching guiding signals, enabling both navigation guidance

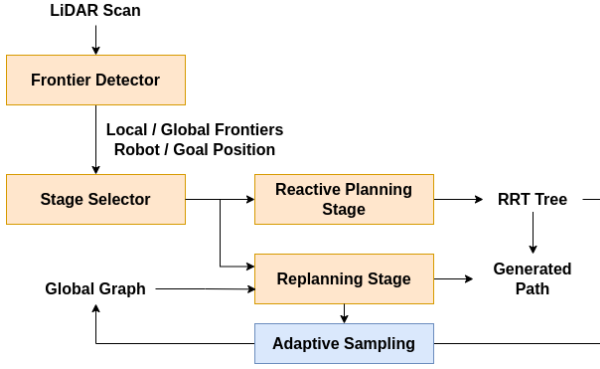


Fig. 2: **The system architecture of FPAS.** The framework consists of four core modules. The frontier detector extracts spatial frontiers to guide navigation. The stage selector dynamically determines when to transition between planning stages. The reactive planning stage generates local paths into unobserved regions within the sensor range. Finally, the replanning stage backtracks from dead-ends and utilizes adaptive sampling to efficiently generate the global graph.

and adaptive management of the global graph. The framework consists of two stages: a reactive planning stage that advances into unobserved regions, and a replanning stage that backtracks to a promising subgoal when encountering dead-ends or inefficient paths. The transition between these two stages is determined by a stage selector. The remainder of this section is organized as follows. Subsection III-A describes the stage selector, while Subsections III-B and III-C detail the reactive planning stage and the replanning stage, respectively. Figure 2 provides an illustration of the overall system.

A. Stage Selector

The role of the stage selector is to choose whether to use reactive planning or replanning. To make this decision, the stage selector leverages frontier information updated at each time step. First, the frontier detector detects and updates the frontier that borders the free space and unknown areas within the sensor range at each time step. The detected frontier is first classified as a local frontier and then reclassified as a global frontier once it moves out of the sensor range. We denote these as F^{local} and F^{global} , respectively. The stage selector determines which stage to enter by comparing F^{local} and F^{global} . If there is no F^{local} available, which can occur when there is no further reachable area left to explore (such as in a dead-end situation), the stage selector immediately selects the replanning stage. Otherwise, it selects the local frontier $\hat{F}^{local}$ closest to the goal g , and checks whether any global frontier F_i^{global} is closer to g than $\hat{F}^{local}$. To prioritize the reactive planning stage and prevent frequent switching between stages, we compare $\hat{F}^{local}$ and F^{global} with a margin ($\beta_1, \beta_2 > 0$) for the distance from the vehicle and the distance to the goal, respectively. If any of the F_i^{global} satisfies these conditions, the stage selector chooses the replanning stage. The overall process is described in

Algorithm 1.

Algorithm 1 Stage Selector

```

1: Input: robot position,  $g$ ,  $F^{local}$ ,  $F^{global}$ 
2: if  $F^{local} = \emptyset$  then
3:   return ReplanningStage()
4: end if
5:  $\hat{F}^{local} \leftarrow \arg \min_{F_i^{local} \in F^{local}} \text{dist}(F_i^{local}, g)$ 
6:  $SetReplan \leftarrow \text{false}$ 
7: for  $F_i^{global} \in F^{global}$  do
8:   if  $\text{dist}(\hat{F}^{local}, \text{robot}) + \beta_1 < \text{dist}(F_i^{global}, \text{robot})$  then
9:     if  $\text{dist}(\hat{F}^{local}, g) + \beta_2 > \text{dist}(F_i^{global}, g)$  then
10:       $SetReplan \leftarrow \text{true}$ 
11:     break;
12:   end if
13: end for
14: if  $SetReplan$  then
15:   return ReplanningStage()
16: else
17:   return ReactivePlanningStage()
18: end if

```

Algorithm 2 ReactivePlanningStage()

```

1: Input: robot position,  $g$ ,  $F^{local}$ 
2:  $\mathcal{N} \leftarrow \text{DynamicRRT}()$ 
3:  $\mathcal{P} \leftarrow \text{ExtractPaths}(\mathcal{N})$ 
4: for all  $P_i \in \mathcal{P}$  do
5:   compute  $I(P_i)$  using Eq. 1
6:   Compute  $GoalDist(P_i)$  using Eq. 2
7:   Compute  $Obj(P_i)$  using Eq. 4
8: end for
9:  $\hat{P}_i \leftarrow \text{best path using Eq. 5}$ 
10: return  $\hat{P}_i$ 

```

B. Reactive Planning Stage

In the reactive planning stage, local paths are determined within the sensor range using a dynamically-expanded RRT [8]. A tree is iteratively generated and expanded within the local perception field. Initially, the RRT is rooted at the vehicle's current position. From the second iteration onward, the existing tree is preserved while new nodes are added via random sampling within the sensor range, facilitating efficient local navigation. We prioritize sampling near $\hat{F}^{local}$ obtained from the stage selector to increase the efficiency of sampling. During this process, nodes that are too close to the existing tree are excluded, and we use OctoMap [22] to determine the traversability between nodes. Here, the OctoMap is updated as the environment is explored. The nodes comprising the tree are denoted as $n_i \in \mathcal{N}$. The position of the vehicle is denoted as n_0 , and we define the path from n_0 to n_i within the tree as $P_i \in \mathcal{P}$. Additionally, n_i^j represents the j -th node that composes P_i .

The path selection balances (i) the amount of new environmental information that can be acquired and (ii) progress toward the final goal g . The amount of new environmental information is quantified as an information gain [21], which we denote as $Gain(\cdot)$. This information gain is calculated by determining the number of newly perceived voxels from the node n_i^j . The total information gain of a candidate path P_i is then defined as follows:

$$I(P_i) = \sum_{n_i^j \in P_i} Gain(n_i^j). \quad (1)$$

The cost of the goal distance $GoalDist(P_i)$ is calculated as the sum of the length of the path and the distance from the last node of the path to the goal.

$$GoalDist(P_i) = PathLength(P_i) + dist(n_i, g). \quad (2)$$

As the vehicle approaches the goal, $GoalDist(P_i)$ decreases, resulting in $I(P_i)$ becoming the dominant factor in the objective function. This may lead to $GoalDist(P_i)$ being disregarded. To balance $GoalDist(P_i)$ and $I(P_i)$, we introduce a sigmoid-based scaling function to dynamically reduce the impact of information gain as the vehicle gets closer to the goal. This function, denoted as $S(x)$, is defined as follows:

$$S(x) = k_1 \cdot \left(\frac{1}{1 + e^{-x/k_2}} - \frac{1}{2} \right), \quad (3)$$

where k_1 and k_2 are scaling constants. Combining these terms, we define the final objective as:

$$Obj(P_i) = I(P_i) \cdot S(dist(n_0, g)) - GoalDist(P_i), \quad (4)$$

In the reactive planning stage, we consider only paths that provide information gain to prevent the vehicle from moving towards already explored areas. To achieve this, we exclude those with zero information gain from consideration. We select the path with the highest objective value among the paths:

$$\hat{P}_i = \arg \max_{P_i \in \mathcal{P}} Obj(P_i) \quad \text{s.t. } I(P_i) \neq 0. \quad (5)$$

Overall, the reactive planning stage drives the vehicle toward a promising local frontier by selecting information-rich paths while maintaining goal-directed progress. This reactive planning procedure is summarized in Algorithm 2.

C. Replanning Stage with Adaptive Sampling

Upon entering the replanning stage, the primary objective is to evaluate and select an appropriate subgoal for backtracking. Specifically, the previously accumulated global frontiers F^{global} are considered as subgoal candidates for a new search. We utilize the following formula to obtain the target subgoal $\hat{F}^{global}$:

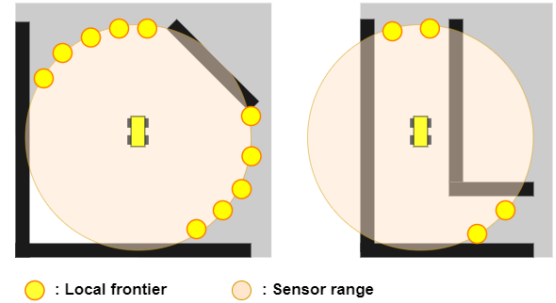
$$\hat{F}^{global} = \arg \min_{F_i^{global} \in F^{global}} dist(F_i^{global}, g). \quad (6)$$

Algorithm 3 ReplanningStage()

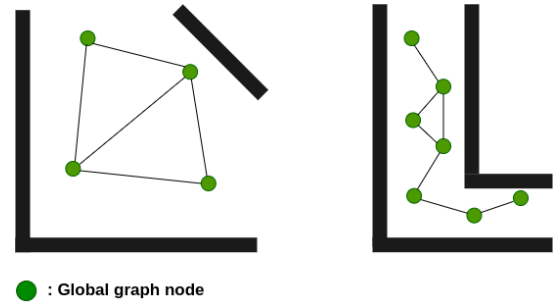
```

1: Input:  $\mathcal{G}, \mathcal{P}, robot, g, F^{local}, F^{global}$ 
2:  $N_{local} \leftarrow |F^{local}|$ 
3: Compute  $Open(N_{local})$  using Eq. 7
4: for all  $P_i \in \mathcal{P}$  do
5:   if  $I(P_i) > 0$  then
6:     for all  $n_i^j \in P_i$  do
7:       Update  $\mathcal{G}$  using adaptive sampling s.t. Eq. 8
8:     end for
9:   end if
10: end for
11: Find best global frontier  $\hat{F}^{global}$  using Eq. 6
12: return  $DijkstraPath(\mathcal{G}, robot, \hat{F}^{global})$ 

```



(a) Detected local frontiers



(b) Adaptively sampled global graph

Fig. 3: (a) Local frontiers (yellow dots) are detected at the boundary between the sensor range (orange circle) and the unknown area. The number of local frontiers increases in open areas (left), while fewer local frontiers are found in narrow passages (right). (b) Openness is measured by the number of local frontiers detected. Nodes from the reactive planning stage are adaptively sampled based on the degree of openness of the area and added to the global graph (green dots).

The environment information acquired during the reactive planning stage is stored as a topological map, denoted as the global graph $\mathcal{G}$. While a straightforward approach would be to retain all nodes generated by the local trees, the continuous accumulation of nodes during exploration inevitably leads to an unbounded increase in graph size. This unbounded growth imposes significant computational burdens on path searching and graph maintenance. To mitigate this bottleneck, we propose an adaptive sampling method that dynamically adjusts the graph's node density, ensuring the efficient construction and long-term management of the global graph. Candidate nodes for updating the global graph are obtained from the reactive planning paths P_i with nonzero information gain ($I(P_i) > 0$). Excluding paths with no information gain prevents oversampling of already explored areas.

We determine the node density based on whether a given region is an open space or a narrow passage. We term this spatial characteristic “openness” and evaluate it using local frontiers, F^{local} . As illustrated in Figure 3, a greater number of local frontiers tends to be detected in expansive open areas compared to confined spaces. Therefore, letting N_{local} denote the number of detected F^{local} and N_{max} denote the maximum possible number of detectable local frontiers, the openness metric can be formulated as follows:

$$Open(N_{local}) = \frac{N_{local}}{N_{max}}. \quad (7)$$

$Open(\cdot)$ has a range of 0 to 1, and the closer it is to 1, the more likely the region is to be an open area. We utilize this openness to determine the distance between nodes during the process of updating the global graph and adjust the density of the global graph accordingly.

Finally, the decision of whether to add candidate nodes to the global graph is made based on their distance from existing nodes in the graph and their traversability. To do this, we first find the node n_g in the global graph that is closest to the node we want to add. Then, we check if the distance between the two nodes satisfies the following condition:

$$\begin{aligned} dist(n_i^j, n_g) &> \delta_{min} \cdot (1 + Open(N_{local})), \\ dist(n_i^j, n_g) &< \delta_{max} \cdot (1 + Open(N_{local})), \end{aligned} \quad (8)$$

where δ_{min} and δ_{max} represent the minimum and maximum distance thresholds, respectively. This condition ensures that higher openness can lead to lower density by increasing the distance between nodes in global graph. If these conditions are satisfied, edges are generated between nearby nodes in the global graph through traversability checks. To find the path to reach the subgoal, we first identify the node closest to the subgoal and the node closest to the vehicle within the global graph. Then, we use Dijkstra's algorithm to find the path between these two nodes. The replanning procedure is detailed in Algorithm 3.

IV. EXPERIMENTS

Our experiments aim to verify whether the FPAS framework can generate near-optimal paths while minimizing

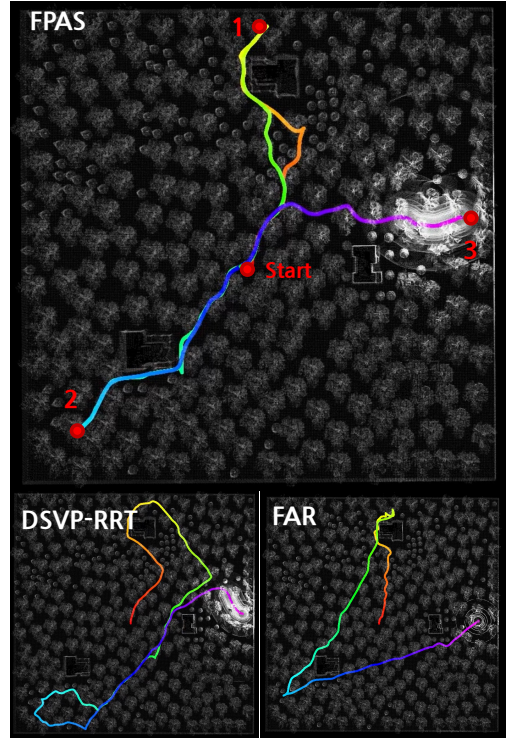


Fig. 4: Vehicle trajectory in a forest environment. Color of the trajectory changes according to the sequence of time in which the path was traversed. Three goals are given sequentially.

computational costs in large-scale, unknown environments. We designed a sequential multi-goal reaching task to evaluate the balance between exploration and exploitation. This approach explicitly demonstrates the planner's efficiency in maintaining and reusing the global graph over extended periods, which a single-goal scenario cannot adequately capture. To validate FPAS across diverse terrains, we conducted simulations in three environments from the Autonomous Exploration Development Environment [23]: Forest, Tunnel, and Garage. The Forest environment predominantly consists of open spaces populated with numerous trees and a few small buildings. The Tunnel environment features a maze-like structure composed of narrow passages, which frequently induces dead-ends and heavily tests the planner's replanning capabilities. The Garage environment presents a complex mixture of both open spaces and narrow passages, making it the most suitable setting to evaluate the adaptive density control mechanism, a core feature of FPAS. In all simulations, the vehicle was equipped with a LiDAR sensor configured with a maximum detection range of 30 meters, and the vehicle's speed was set to 2m/s.

To objectively benchmark the performance of FPAS, we compared it against two competitive baselines. The first baseline is a modified Dynamically-expanded RRT. Because the standard RRT is unsuitable for large-scale unknown environments, we adopted the dynamically-expanded RRT structure used in the DSVP algorithm [8]. To ensure a

TABLE I: Average processing time and travel distance in forest environment

	DSVP-RRT	FAR	FPAS (ours)
Goal 1	4.61	48.14	4.42
Goal 2	5.73	83.75	2.76
Goal 3	6.25	87.81	2.23
Average	5.53	73.23	3.14

(a) Average processing time (ms)

	DSVP-RRT	FAR	FPAS (ours)
Goal 1	90.32	94.52	92.17
Goal 2	229.69	149.07	170.77
Goal 3	197.09	138.53	173.75
Total	517.1	382.12	436.69

(b) Travel distance (m)

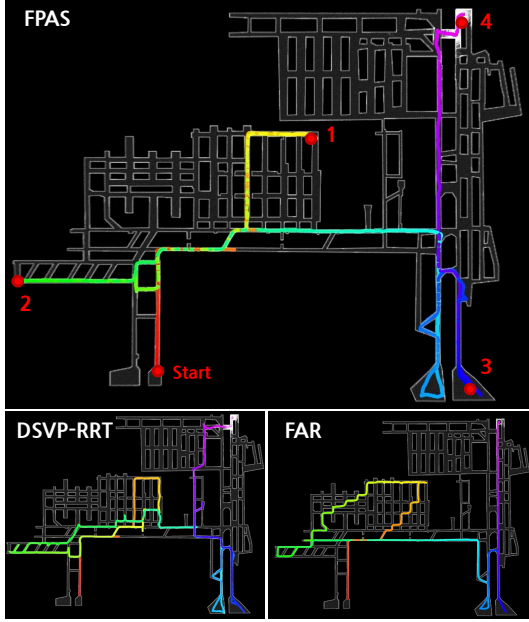


Fig. 5: Vehicle trajectory in a tunnel environment. Four goals are given sequentially.

controlled comparison, its path selection objective was modified to be identical to that of FPAS. Hereafter, we refer to this baseline as DSVP-RRT. The second baseline is the FAR Planner, a state-of-the-art method for goal-reaching in unknown environments utilizing a visibility graph. We evaluated all methods using travel distance and average processing time (computation time per simulation step). Results are averaged over 10 independent runs per environment. All evaluations were performed on a system equipped with a 5.2GHz Intel Core i9 CPU.

A. Results

The results in the forest environment are presented in Figure 4 and Table I. While FAR achieved the shortest travel distance due to the environment’s straight-line feasibility, our method prioritizes acquiring new environmental information during reactive planning, making a slight trade-off in distance. However, FPAS drastically outperformed FAR

TABLE II: Average processing time and travel distance in tunnel environment

	DSVP-RRT	FAR	FPAS (ours)
Goal 1	12.31	7.22	14.52
Goal 2	11.42	10.56	11.25
Goal 3	16.05	12.67	11.23
Goal 4	14.50	10.58	11.86
Average	13.57	10.26	12.22

(a) Average processing time (ms)

	DSVP-RRT	FAR	FPAS (ours)
Goal 1	259.48	234.89	259.65
Goal 2	310.79	309.26	306.07
Goal 3	686.41	644.49	619.38
Goal 4	385.87	247.59	310.31
Total	1642.55	1436.23	1495.41

(b) Travel distance (m)

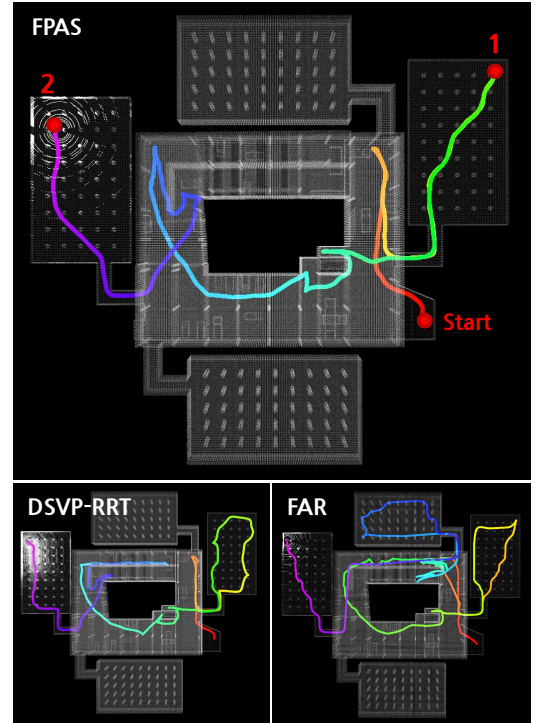


Fig. 6: Vehicle trajectory in a garage environment. Two goals are given sequentially.

in computational efficiency. FAR suffered a heavy burden from polygonizing non-standardized obstacles (e.g., trees) and managing an increased number of edges in wide areas. Consequently, FPAS achieved an average processing time of 3.14ms, representing a significant reduction compared to FAR (73.23ms) and DSVP-RRT (5.53ms), effectively validating the efficiency of our adaptive sampling approach.

In the tunnel environment (Figure 5, Table II), the maze-like structure necessitates robust replanning and effective use of prior information. Here, FPAS demonstrated superior performance over DSVP-RRT in both processing time (12.22ms vs. 13.57ms) and total travel distance (1495.41m vs. 1642.55m). Unlike the forest, the tunnel’s narrow, elongated

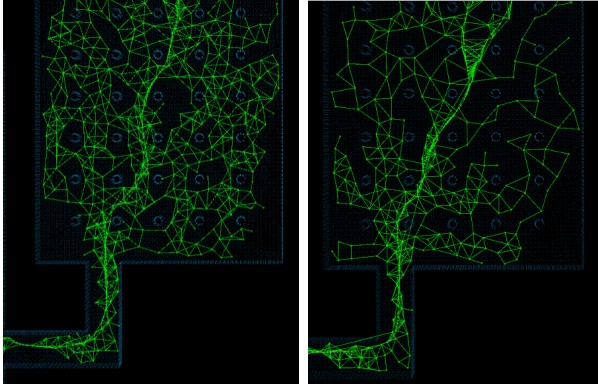
TABLE III: Average processing time and travel distance in garage environment

	DSVP-RRT	FAR	FPAS (ours)
Goal 1	7.57	18.49	5.35
Goal 2	8.94	28.42	4.96
Average	8.26	23.46	5.16

(a) Average processing time (ms)

	DSVP-RRT	FAR	FPAS (ours)
Goal 1	175.13	177.32	165.52
Goal 2	470.52	811.63	353.43
Total	645.65	988.95	518.95

(b) Travel distance (m)



(a) W/o adaptive sampling (b) Adaptively sampled

Fig. 7: Visualization of the global graph, both without (a) and with (b) applying adaptive sampling, in the garage environment.

gated passages facilitate straightforward visibility graph updates for FAR, yielding a slightly lower processing time (10.26ms) and distance (1436.23m). Nevertheless, FPAS remained highly competitive, proving its robust adaptability and efficiency even in heavily constrained spaces.

Finally, the garage environment (Figure 6, Table III) highlights the planner’s ability to handle complex, mixed topographies. Similar to the forest, FPAS achieved the lowest average processing time (5.16ms) compared to FAR (23.46ms) and DSVP-RRT (8.26ms). Furthermore, to reach the second goal located inside a room, the baselines failed to utilize a critical narrow passage due to their greedy heuristics, resulting in severe detours. In contrast, FPAS successfully navigated back to the passage via efficient replanning. This led to a substantial reduction in travel distance, with FPAS covering only 518.95m compared to FAR’s 988.95m and DSVP-RRT’s 645.65m. Ultimately, these evaluations confirm that FPAS maintains consistent, high-speed, and robust performance regardless of varying environmental characteristics.

B. Ablation study

An ablation study was conducted in the garage environment to compare the performance improvement of applying adaptive sampling. Figure 7 visualizes the global graph

TABLE IV: Ablation study

	w/o Adaptive Sample	Adaptively Sampled
Number of nodes	2633	1839
Avg. processing time (ms)	7.01	5.16
Travel dist (m)	496.39	518.95

according to the application of adaptive sampling. As shown in Figure 7b, the nodes of the global graph are densely located near the narrow passage located at the bottom, while the top open region has sparsely located nodes. Using adaptive sampling, we were able to reduce the number of nodes required to construct the global graph from 2633 to 1839, leading to a reduction in average processing time from 7.01ms to 5.16ms. However, we observed a slight increase in travel distance from 496.39m to 518.95m when utilizing the adaptively sampled graph. This may be attributed to the fact that the adaptive sampling process used to sparsify the graph can result in the loss of information needed for the shortest path generation. We will leave this issue as future work for our method.

V. CONCLUSION

In this paper, we presented Frontier-based Path Planning with Adaptive Sampling (FPAS), a novel framework designed to achieve efficient goal-reaching in large-scale, unknown environments. By reinterpreting the frontier concept, FPAS effectively balances forward exploration into unobserved regions with optimal backtracking when encountering dead-ends. Furthermore, we introduced an adaptive sampling mechanism based on a frontier-derived openness metric. This approach dynamically adjusts the density of the global graph, significantly alleviating computational and memory burdens by employing sparse nodes in open spaces while maintaining dense sampling in narrow passages.

Despite its advantages, the current framework presents limitations in path optimality and environmental scalability. Deliberate graph sparsification in open areas can cause minor deviations from the absolute shortest paths. Additionally, its reliance on 2D/2.5D representations limits performance in fully 3D environments (e.g., UAVs) or for vehicles with non-holonomic constraints. To address these issues, our future work will refine the adaptive sampling process to enforce minimum path length constraints, ensuring near-optimality alongside computational efficiency. Furthermore, we plan to extend the openness metric to 3D voxel spaces and incorporate kinodynamic constraints into trajectory generation, expanding the framework’s applicability to more complex robotic systems.

REFERENCES

- [1] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, “Path planning for autonomous vehicles in unknown semi-structured environments,” *The international journal of robotics research*, vol. 29, no. 5, pp. 485–501, 2010.
- [2] L. Yang, J. Qi, D. Song, J. Xiao, J. Han, and Y. Xia, “Survey of robot 3d path planning algorithms,” *Journal of Control Science and Engineering*, vol. 2016, no. 1, p. 7426913, 2016.

- [3] J. Delmerico, S. Mintchev, A. Giusti, B. Gromov, K. Melo, T. Horvat, C. Cadena, M. Hutter, A. Ijspeert, D. Floreano *et al.*, "The current state and future outlook of rescue robotics," *Journal of Field Robotics*, vol. 36, no. 7, pp. 1171–1191, 2019.
- [4] V. Roberge, M. Tarbouchi, and G. Labonté, "Fast genetic algorithm path planner for fixed-wing military uav using gpu," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 54, no. 5, pp. 2105–2117, 2018.
- [5] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [6] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Proceedings of the 1994 IEEE international conference on robotics and automation*. IEEE, 1994, pp. 3310–3317.
- [7] F. Yang, C. Cao, H. Zhu, J. Oh, and J. Zhang, "Far planner: Fast, attemptable route planner using dynamic visibility update," in *2022 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2022, pp. 9–16.
- [8] H. Zhu, C. Cao, Y. Xia, S. Scherer, J. Zhang, and W. Wang, "Dsvp: Dual-stage viewpoint planner for rapid exploration by dynamic expansion," in *2021 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2021, pp. 7623–7630.
- [9] S. Koenig, M. Likhachev, and D. Furcy, "Lifelong planning a*," *Artificial Intelligence*, vol. 155, no. 1-2, pp. 93–146, 2004.
- [10] S. Koenig and M. Likhachev, "D* lite," in *Eighteenth national conference on Artificial intelligence*, 2002, pp. 476–483.
- [11] R. Cao, Z. Yang, R. Song, Z. Meng, R. Wang, and W. Zhang, "Mpp: Multiscale path planning for ugv navigation in semi-structured environments," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 5497–5504.
- [12] J. J. Kuffner and S. M. LaValle, "Rrt-connect: An efficient approach to single-query path planning," in *Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065)*, vol. 2. IEEE, 2000, pp. 995–1001.
- [13] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Informed rrt*: Optimal sampling-based path planning focused via direct sampling of an admissible ellipsoidal heuristic," in *2014 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2014, pp. 2997–3004.
- [14] D. J. Webb and J. v. d. Berg, "Kinodynamic rrt*: Optimal motion planning for systems with linear differential constraints," *arXiv preprint arXiv:1205.5088*, 2012.
- [15] M. Otte and E. Frazzoli, "Rrtx: Asymptotically optimal single-query sampling-based motion planning with quick replanning," *The International Journal of Robotics Research*, vol. 35, no. 7, pp. 797–822, 2016.
- [16] X. Dong, Y. Wang, C. Fang, K. Ran, and G. Liu, "Fhq-rrt*: An improved path planning algorithm for mobile robots to acquire high-quality paths faster," *Sensors*, vol. 25, no. 7, p. 2189, 2025.
- [17] D. Devaurs, T. Siméon, and J. Cortés, "Parallelizing rrt on large-scale distributed-memory architectures," *IEEE Transactions on Robotics*, vol. 29, no. 2, pp. 571–579, 2013.
- [18] L. Jaillet, A. Yershova, S. M. La Valle, and T. Siméon, "Adaptive tuning of the sampling domain for dynamic-domain rrts," in *2005 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2005, pp. 2851–2856.
- [19] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 2002.
- [20] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, "Receding horizon" next-best-view" planner for 3d exploration," in *2016 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2016, pp. 1462–1468.
- [21] T. Dang, F. Mascarich, S. Khattak, C. Papachristos, and K. Alexis, "Graph-based path planning for autonomous robotic exploration in subterranean environments," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 3105–3112.
- [22] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [23] C. Cao, H. Zhu, F. Yang, Y. Xia, H. Choset, J. Oh, and J. Zhang, "Autonomous exploration development environment and the planning algorithms," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 8921–8928.